



Lab Report 2

Experiment on Fixed-Point Grasping and Handling with mechArm270

Group Members

Cao Jingwen 24020036005
Dong Xiaorui 24020036015
Deng Wanqian 24020036013
Liu Wenshuo 24020036044

Integrated Robotics Group Project

Group 5

2026-09-14

Abstract

This report presents the design, implementation, and validation of a fixed-point pick-and-place task using the Elephant Robotics **mechArm270** six-axis manipulator, conducted in both the Gazebo / Ignition simulation environment and on the physical robotic arm. A simulation scene containing the arm, gripper, worktable, and target object is first constructed; fixed pick point *A*, place point *B*, and a safe height are then specified. The motion sequence—homing, approach, descent, grasping, lifting, transfer, release, and homing—is implemented with ROS 2 and **ros2_control** through the **FollowJointTrajectory** interface. In simulation, five consecutive trials and an unreachable-target test are completed; on the physical robot, five pick-and-place trials succeed with a 100% success rate and no collisions, and an out-of-range joint target is correctly rejected. The results validate the reliability, repeatability, and safety of the implemented workflow.

Keywords: fixed-point grasping; pick-and-place; ROS 2; Gazebo; inverse kinematics; physical robot; **mechArm270**

Contents

1	Objectives	3
1.1	Overview	3
1.2	Experimental Tasks	3
2	Experimental Platform and System Composition	3
2.1	Software and Hardware Environment	3
2.2	ROS 2 Control Architecture	4
3	Fixed-Point Pick-and-Place Method	4
3.1	Fixed Points and Safe Height	4
3.2	Inverse Kinematics and Safety Protection	4
4	Gazebo Simulation Experiment	4
4.1	Simulation Scene Construction	4
4.2	Execution of Fixed-Point Pick-and-Place Task	5
5	Simulation Results and Analysis	6
5.1	Five Consecutive Grasping Tests	6
5.2	Trajectory Recording	7
5.3	Abnormal Target Test	7
6	Physical Robotic Arm Experiment	8
6.1	Physical Robot Driver and Task Execution	8
6.2	Joint State Feedback	9
7	Physical Robot Test Results	9
7.1	Five Consecutive A-to-B Pick-and-Place Trials	9
7.2	Joint Limit Safety Test	9
8	Program and Engineering Design	10
8.1	Overall System Architecture	10
8.2	Principles of Simulation Program	11
8.2.1	Task Node	11

8.2.2	Launch File and Controller Configuration	12
8.3	Principles of Physical Robot Program	13
8.3.1	Driver Node <code>real_driver.py</code> and TCP Communication	13
8.3.2	Teaching Points and Continuous Pick-and-Place Script	13
9	GitHub Project Management	14
10	Problem Analysis and Limitations	14
11	Experiment Conclusion	14
12	GitHub Repository and Commit History	15

1 Objectives

1.1 Overview

The primary objective of this experiment is to implement the fixed-point pick-and-place task using the **mechArm270** 6-DOF robotic arm, and to validate the task on both the Gazebo / Ignition simulation environment and the physical robotic arm platform. First, a simulation scene containing the robotic arm, gripper, workbench, and target object is constructed. In simulation, the robotic arm performs a sequence of motions including homing, approaching the fixed point, descending, grasping, lifting, transporting, releasing, and returning to the home position. After successful simulation verification, the identical ROS 2 control logic is migrated to the physical robotic arm.

No visual localization is adopted in this experiment. Both the grasp point A and the placement point B are predefined fixed positions. The complete task sequence is: **Home** $\rightarrow$ safe position above point A $\rightarrow$ descend to grasp position $\rightarrow$ gripper close $\rightarrow$ lift up $\rightarrow$ move to above point B $\rightarrow$ descend $\rightarrow$ gripper open $\rightarrow$ return to **Home**.

1.2 Experimental Tasks

1. Construct a simulation scene consisting of the robotic arm, gripper, worktable, and target object.
2. Set fixed grasp point A , fixed placement point B , and safe heights for the pick-and-place procedure.
3. Implement the motion sequence: home $\rightarrow$ move to above the grasp point $\rightarrow$ descend $\rightarrow$ grasp $\rightarrow$ lift $\rightarrow$ move to the placement point $\rightarrow$ release $\rightarrow$ home.
4. Control the robotic arm via ROS 2 nodes or Actions, and publish the robot state.
5. Launch the simulation, control, and state nodes with a single launch file.
6. Detect conditions such as unreachable targets, no inverse-kinematics solution, and joint-limit violation, and trigger a safe stop accordingly.

2 Experimental Platform and System Composition

2.1 Software and Hardware Environment

The software and hardware environment of the experiment is summarized in Table 1.

Table 1: Software and hardware environment.

Item	Configuration
Operating System	Ubuntu 22.04
Robot Middleware	ROS 2 Humble
Simulation Platform	Gazebo / Ignition Fortress
Motion and Control	MoveIt 2, <code>ros2_control</code> , <code>FollowJointTrajectory</code>
Physical Robot Controller	NVIDIA Jetson Orin
Robotic Arm	Elephant Robotics mechArm270 with gripper
Task Type	Fixed-point pick-and-place from A to B

2.2 ROS 2 Control Architecture

The task node generates motion targets for the robotic arm according to the predefined grasp and placement positions. It sends joint trajectories to the robot controller via the ROS 2 `FollowJointTrajectory` interface. The current joint states of the robotic arm are fed back through the `joint_states` topic. The high-level task pipeline remains identical for both simulation and physical-robot tests. The main difference lies in the low-level control interface, which switches from the Gazebo simulation controller to the driver of the physical robotic arm.

3 Fixed-Point Pick-and-Place Method

3.1 Fixed Points and Safe Height

The fixed-point strategy is adopted for grasping in this experiment. In simulation tests, grasp point A is $(0.10, 0.02, 0.7635)$ m, and placement point B is $(0.07, -0.08, 0.7635)$ m. Before horizontal movement, the robotic arm lifts to a safe height to reduce the risk of collision among the gripper, target object, and workbench. The key parameters are listed in Table 2.

Table 2: Fixed points and safe height.

Parameter	Value
Grasp point A	$[0.10, 0.02, 0.7635]$ m
Placement point B	$[0.07, -0.08, 0.7635]$ m
Safe height	0.065 m
End-effector offset	0.063 m
Gripper open / close	0.0 / -0.44 rad
Number of repeated tests	5

3.2 Inverse Kinematics and Safety Protection

The robotic arm calculates the corresponding joint angles according to the target end-effector pose. Before trajectory transmission, the program verifies whether the target joint angles exceed the joint limits defined in the URDF file. If the inverse kinematics solution does not exist, the target is unreachable, or joint-angle overflow occurs, the program does not force the robotic arm to keep moving. Instead, it terminates the current task and logs error messages. This design ensures safety during both simulation and physical-robot operation.

4 Gazebo Simulation Experiment

4.1 Simulation Scene Construction

The Gazebo / Ignition scene consists of the `mechArm270` robotic arm, gripper, fixed workbench, and the target object to be grasped. After model loading, the robot joints, gripper, workspace, and the position of the target object are inspected to verify that all models can be displayed normally and accept ROS 2 control commands. Figures 1 and 2 show the simulation scene and the workspace.

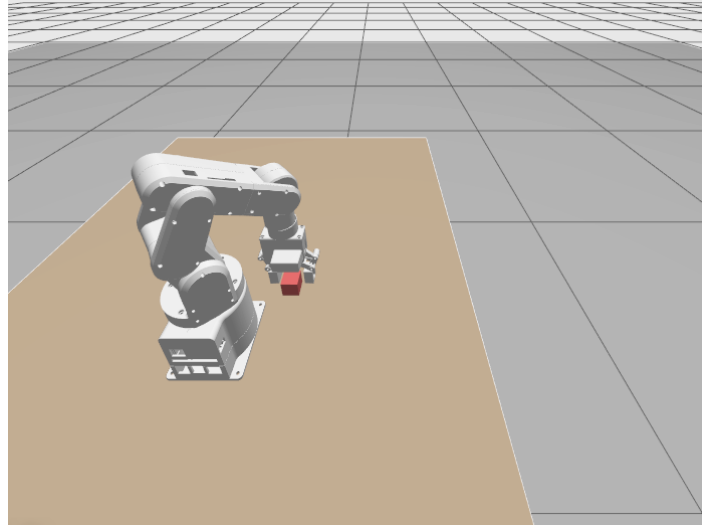


Figure 1: Gazebo / Ignition simulation scene.

```

ation method.
ign-1] [INFO] [1788791695.638593501] [hand_controller]: Controller state will b
published at 50.00 Hz.
ign-1] [INFO] [1788791695.641196139] [hand_controller]: Action status changes w
ll be monitored at 20.00 Hz.
spawner-7] [INFO] [1788791695.688306273] [spawner_hand_controller]: Configured
and activated hand_controller
[INFO] [spawner-7]: process has finished cleanly [pid 45154]
lws_transfer_task-8] [INFO] [1788791697.512033955] [lws_transfer_node]: 正在连
接机械臂控制器...
lws_transfer_task-8] [INFO] [1788791697.513161555] [lws_transfer_node]: 正在计
算各目标位姿...
lws_transfer_task-8] [INFO] [1788791711.477863988] [lws_transfer_node]: 正在确
认目标物体位置...
lws_transfer_task-8] [INFO] [1788791711.704196013] [lws_transfer_node]: 已获取
目标物体位置: [0.1 0.02 0.7625]
lws_transfer_task-8] [INFO] [1788791711.705534598] [lws_transfer_node]: =====
第 1/5 轮搬运任务 (A-B) =====
lws_transfer_task-8] [INFO] [1788791711.707840648] [lws_transfer_node]: [1] 返
回初始姿态
ign-1] [INFO] [1788791711.710155915] [arm_controller]: Received new action goal
ign-1] [INFO] [1788791711.710370292] [arm_controller]: Accepted new action goal
[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)
parameter_bridge-2] [INFO] [1788791712.913826092] [rclcpp]: signal_handler(SIGI
NT/SIGTERM)
robot_state_publisher-3] [INFO] [1788791712.915937510] [rclcpp]: signal_handler
(SIGINT/SIGTERM)
[INFO] [parameter_bridge-2]: process has finished cleanly [pid 44976]

```

Figure 2: Robotic arm model and workspace.

4.2 Execution of Fixed-Point Pick-and-Place Task

Once the task starts, the robotic arm first returns to the home state and opens the gripper. It then moves to the safe position above point *A*. Afterwards, the arm descends to the grasp height, closes the gripper to lift the target object, and moves to the position above point *B*. The arm descends again and opens the gripper to complete the placement, and finally returns to the home position. The execution is shown in Figure 3.

```

nvidia@nvidia-desktop:~/lws/colcon_ws$ echo "==== 异常错误日志 ====="
tail -n 20 ~/lws/grasp_logs/errors.log

echo
echo "==== 当前日志文件 ====="
ls -lh ~/lws/grasp_logs/
==== 异常错误日志 =====
2026-09-07T21:03:39.000578 发送轨迹目标超时 (通信异常)
2026-09-07T22:26:22.325178 目标位置 above_a 无法获得有效关节解, 停止当前任务
2026-09-07T22:26:22.326103 目标不可达 (超出工作半径或无逆解), 任务拒绝启动, 机械臂保持安全停止
2026-09-07T22:40:48.371319 轨迹执行超时
2026-09-07T22:42:33.477173 轨迹执行超时

==== 当前日志文件 =====
total 1008K
-rw-rw-r-- 1 nvidia nvidia 389 Sep 7 22:42 errors.log
-rw-rw-r-- 1 nvidia nvidia 191 Sep 7 22:49 results.csv
-rw-rw-r-- 1 nvidia nvidia 52 Sep 7 22:49 summary.txt
-rw-rw-r-- 1 nvidia nvidia 996K Sep 7 22:49 trajectory.csv

```

Figure 3: Execution of the fixed-point pick-and-place task in simulation.

5 Simulation Results and Analysis

5.1 Five Consecutive Grasping Tests

Five consecutive tests of the fixed-point pick-and-place workflow were carried out in accordance with the experimental acceptance requirements. All five tasks were recorded as successful in the logs, indicating that the robotic arm can stably complete the predefined grasping and handling workflow. The results are listed in Table 3.

Table 3: Results of the five consecutive simulation tests.

Trial No.	Success	X/m	Y/m	Z/m
1	Yes	0.0699	-0.0800	0.7625
2	Yes	0.1041	0.0203	0.7625
3	Yes	0.0701	-0.0799	0.7625
4	Yes	0.1041	0.0207	0.7625
5	Yes	0.0710	-0.0799	0.7625

```

nvidia@nvidia-desktop:~$ echo "===== 机械臂轨迹日志 ====="
echo "轨迹文件总行数："
wc -l ~/lws/grasp_logs/trajectory.csv

echo
echo "===== 前 12 行轨迹数据 ====="
head -n 12 ~/lws/grasp_logs/trajectory.csv
===== 机械臂轨迹日志 =====
轨迹文件总行数：
17368 /home/nvidia/lws/grasp_logs/trajectory.csv

===== 前 12 行轨迹数据 =====
stamp,joint1_to_base,joint2_to_joint1,joint3_to_joint2,joint4_to_joint3,j
o_joint4,joint6_to_joint5,gripper_controller
2.472,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.484,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.496,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.508,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.520,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.532,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.544,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.556,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.568,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.580,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000
2.592,-0.0000,-0.0000,0.0013,-0.0000,-0.0011,0.0000,0.0000

```

Figure 4: Results of the five simulation tests.

5.2 Trajectory Recording

The program continuously records timestamps and joint positions of the robotic arm during operation and saves them to `trajectory.csv`. A total of 17368 lines of trajectory data are recorded in the test logs, which can be used to inspect the robot motion and support subsequent analysis of experimental results.

```

nvidia@nvidia-desktop:~/lws/colcon_ws$ echo "===== 5 次抓取结果 ====="
cat ~/lws/grasp_logs/results.csv

echo
echo "===== 任务总结 ====="
cat ~/lws/grasp_logs/summary.txt
===== 5 次抓取结果 =====
cycle,succcess,object_x,object_y,object_z,detail
1,1,0.0699,-0.0800,0.7625,ok
2,1,0.1041,0.0203,0.7625,ok
3,1,0.0701,-0.0799,0.7625,ok
4,1,0.1041,0.0207,0.7625,ok
5,1,0.0710,-0.0799,0.7625,ok

===== 任务总结 =====
搬运流程结束：共执行 5 轮，完成 5 轮

```

Figure 5: Trajectory and terminal logs.

5.3 Abnormal Target Test

To verify the safety handling capability under abnormal conditions, targets that are unreachable or have no valid inverse-kinematics solution are configured in the experiment. When the program detects that a target cannot be executed, it rejects the current task and logs errors in `errors.log`, instead of continuing to send hazardous trajectories.

```

===== Experiment 3.5 异常测试记录 =====

2026-09-07T21:03:39.000578 发送轨迹目标超时（通信异常）
2026-09-07T22:26:22.325178 目标位置 above_a 无法获得有效关节解，停止当前任务
2026-09-07T22:26:22.326103 目标不可达（超出工作半径或无逆解），任务拒绝启动，机械臂保持安全停止
2026-09-07T22:40:48.371319 轨迹执行超时
2026-09-07T22:42:33.477173 轨迹执行超时

===== 实验日志文件 =====
total 1008K
-rw-rw-r-- 1 nvidia nvidia 389 Sep 7 22:42 errors.log
-rw-rw-r-- 1 nvidia nvidia 191 Sep 7 22:49 results.csv
-rw-rw-r-- 1 nvidia nvidia 52 Sep 7 22:49 summary.txt
-rw-rw-r-- 1 nvidia nvidia 996K Sep 7 22:49 trajectory.csv

```

Figure 6: Simulated abnormal targets and error logs.

6 Physical Robotic Arm Experiment

6.1 Physical Robot Driver and Task Execution

After completing simulation verification, the fixed-point pick-and-place task is migrated to the physical mechArm270. The physical robotic arm runs the ROS 2 driver on Jetson Orin, and the task node continues to send robot motion commands via trajectory control. A relatively conservative motion speed is adopted during physical-robot tests to ensure safety in the debugging process.

```

nvidia@nvidia-desktop:~$ export ROS_DOMAIN_ID=42
source /opt/ros/humble/setup.bash
source ~/lws/colcon_ws/install/setup.bash

ros2 run mecharm_real real_driver \
--ros-args \
--params-file ~/lws/colcon_ws/src/mecharm_real/config/real.yaml
[INFO] [1789289079.130197632] [mecharm_real_driver]: 真机驱动就绪：臂内服务 10.4
2.0.89:9001, 低速模式 max_speed=20

```

Figure 7: Launch of the physical robotic arm driver on Jetson.

```

[INFO] [1789233042.798174403] [real_transfer_ws]: ===== LWS 真机 ROS2 连续5段 A->B->A->B->A->B =====
[INFO] [1789233042.798708996] [real_transfer_ws]: HOME = [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233042.799170253] [real_transfer_ws]: A_SAFE = [-9.49, 42.62, 0.06, -6.15, 7.82, 2.46] deg
[INFO] [1789233042.799910172] [real_transfer_ws]: A_CLEAR = [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233042.800393560] [real_transfer_ws]: A_PICK = [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233042.800893365] [real_transfer_ws]: B_CLEAR = [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233042.801570137] [real_transfer_ws]: B_PLACE = [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233042.802060153] [real_transfer_ws]: executeStatus = 准备执行连续5段逆解动作
[INFO] [1789233042.827589402] [real_transfer_ws]: 当前机械臂姿态 = [-6.32, 44.73, 3.42, -1.93, 57.91, 0.26] deg
[INFO] [1789233042.828197990] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233042.868413756] [real_transfer_ws]: 打开夹爪
[INFO] [1789233042.869332580] [real_transfer_ws]: 移动A_SAFE: [-9.49, 42.62, 0.06, -6.15, 7.82, 2.46] deg
[INFO] [1789233050.517028112] [real_transfer_ws]: 移动A_PICK: [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233050.677385805] [real_transfer_ws]: 闭合夹爪
[INFO] [1789233062.772315460] [real_transfer_ws]: 移动B_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233065.157972460] [real_transfer_ws]: 移动B_PLACE: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233070.177913973] [real_transfer_ws]: 移动B_PLACE: [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233072.540818099] [real_transfer_ws]: 打开夹爪
[INFO] [1789233076.433714855] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233076.760494994] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233080.275821897] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233093.552647633] [real_transfer_ws]: 移动B_PLACE: [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233095.960050533] [real_transfer_ws]: 闭合夹爪
[INFO] [1789233099.833208441] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233100.184445732] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233107.236108141] [real_transfer_ws]: 移动A_PICK: [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233109.586501530] [real_transfer_ws]: 打开夹爪
[INFO] [1789233113.495392143] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233115.805453233] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233121.317643020] [real_transfer_ws]: 移动A_SAFE: [-9.49, 42.62, 0.06, -6.15, 7.82, 2.46] deg
[INFO] [1789233130.754654118] [real_transfer_ws]: 移动A_PICK: [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233135.181790480] [real_transfer_ws]: 闭合夹爪
[INFO] [1789233137.032938424] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233139.391073850] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233144.653160777] [real_transfer_ws]: 移动B_PLACE: [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233147.007498259] [real_transfer_ws]: 打开夹爪
[INFO] [1789233150.932523237] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233153.291407435] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233160.772840264] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233166.212464564] [real_transfer_ws]: 移动B_PLACE: [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233170.573285966] [real_transfer_ws]: 闭合夹爪
[INFO] [1789233174.462208099] [real_transfer_ws]: 移动A_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233176.814538061] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233182.606897581] [real_transfer_ws]: 移动A_PICK: [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233184.403017144] [real_transfer_ws]: 打开夹爪
[INFO] [1789233188.335428260] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233190.497997927] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233198.178074227] [real_transfer_ws]: 移动A_SAFE: [-9.49, 42.62, 0.06, -6.15, 7.82, 2.46] deg
[INFO] [1789233200.660314416] [real_transfer_ws]: 移动A_PICK: [-9.49, 48.86, 5.36, -6.15, 7.82, 2.46] deg
[INFO] [1789233206.914081809] [real_transfer_ws]: 闭合夹爪
[INFO] [1789233211.584362022] [real_transfer_ws]: 移动A_CLEAR: [-9.49, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233214.867293227] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233219.519426413] [real_transfer_ws]: 移动B_PLACE: [25.0, 48.51, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233221.875243930] [real_transfer_ws]: 打开夹爪
[INFO] [1789233225.753671977] [real_transfer_ws]: 移动B_CLEAR: [25.0, 38.0, 7.47, -6.15, 7.82, 2.46] deg
[INFO] [1789233228.691323844] [real_transfer_ws]: 移动HOME [-6.76, 43.94, 2.02, -1.75, 58.44, -0.08] deg
[INFO] [1789233235.571841318] [real_transfer_ws]: ===== ROS2 连续5段逆解动作完成 =====

real 302.076s
user 604.006s
sys 600.312s

```

Figure 8: Fixed-point pick-and-place experiment with the physical robotic arm.

6.2 Joint State Feedback

The command `ros2 topic echo /joint_states --once` can be used to read the current positions of the six robot joints and the gripper controller. This confirms that the state-feedback link between the physical robot driver and ROS 2 works properly.

```
nvidia@nvidia-desktop:~/lws$ ros2 topic echo /joint_states --once
header:
  stamp:
    sec: 1789291668
    nanosec: 639833201
  frame_id: ''
name:
- joint1_to_base
- joint2_to_joint1
- joint3_to_joint2
- joint4_to_joint3
- joint5_to_joint4
- joint6_to_joint5
- gripper_controller
position:
- 0.12409290981679684
- 0.6702064327658225
- 0.2822197400474831
- -0.03979350694547071
- 1.006182313774731
- -0.06283185307179587
- 0.0
velocity: []
effort: []
---
```

Figure 9: `/joint_states` state feedback on the physical robot.

7 Physical Robot Test Results

7.1 Five Consecutive A-to-B Pick-and-Place Trials

A total of five independent fixed-point grasping and placement tests were carried out on the physical robotic arm. The recorded results show that all five grasping and placement operations succeeded with no collisions, achieving a success rate of 100%, as listed in Table 4.

Table 4: Results of the five consecutive physical-robot pick-and-place trials.

Trial No.	Grasp	Placement	Collision	Time/s
1	Success	Success	None	36.026
2	Success	Success	None	36.038
3	Success	Success	None	36.060
4	Success	Success	None	35.914
5	Success	Success	None	35.954
Summary	5/5	100%	0	Avg. 35.998

The runtime of all five trials was approximately 36 s, with the fastest at 35.914 s and the slowest at 36.060 s. The small time fluctuation indicates good repeatability of task execution under the current parameters and low-speed safety settings.

7.2 Joint Limit Safety Test

An invalid target of 2.967 rad was manually sent to `joint1_to_base` to verify joint-limit protection. After detecting that this target exceeds the joint range defined in the URDF, the controller

returns `error_code = -5` and terminates the task with an `ABORTED` status. The robotic arm did not execute this hazardous target, which proves that the joint-limit protection works effectively.

```
Goal accepted with ID: 223e03b5fb174dfcab36389bacaf66f9

Result:
  error_code: -5
  error_string: 关节 joint1_to_base 目标 2.967 超 URDF 限位

Goal finished with status: ABORTED
===== AFTER =====
header:
  stamp:
    sec: 1789289193
    nanosec: 936350066
  frame_id: ''
name:
- joint1_to_base
- joint2_to_joint1
- joint3_to_joint2
- joint4_to_joint3
- joint5_to_joint4
- joint6_to_joint5
- gripper_controller
position:
- 0.12566370614359174
- 0.6702064327658225
- 0.2822197400474831
- -0.044331363000655974
- 0.998502865065956
- -0.06283185307179587
- 0.0
velocity: []
effort: []
---
```

Figure 10: Rejection of an out-of-range joint target on the physical robot.

8 Program and Engineering Design

In terms of engineering structure, this experiment separately manages simulation and `real_robot` modules, while both reuse the same high-level task pipeline and ROS 2 control interfaces. The implementation principles of the core code are explained from three aspects: overall system architecture, simulation program, and physical-robot program.

8.1 Overall System Architecture

The entire system adopts a layered design. At the top layer lies the task node, which generates joint motion targets according to fixed grasp point A and placement point B , controls gripper opening/closing, and records trajectories and results. The middle layer consists of unified ROS 2 control interfaces: the task node sends trajectories only via the `FollowJointTrajectory` action, reads joint states through the `/joint_states` topic, and triggers a software emergency stop via the `/soft_stop` topic. The bottom layer is the controller. During simulation, the `JointTrajectoryController` from `ros2_control` directly drives the virtual robotic arm in Gazebo / Ignition. For physical-robot tests, the self-developed `real_driver` action server receives targets through the same interface, communicates with `arm_server.py` running on the Raspberry Pi inside the arm via TCP, and finally calls `pymycobot` to drive the physical robotic arm. Benefiting from the unified interface, no modification is required for the task-node code when switching from simulation to the physical robot; only the underlying controller and parameter files need replacement. This satisfies the acceptance requirement that “switching to the physical robot only modifies device, communication, and position parameters”. The architecture is shown in Figure 11.

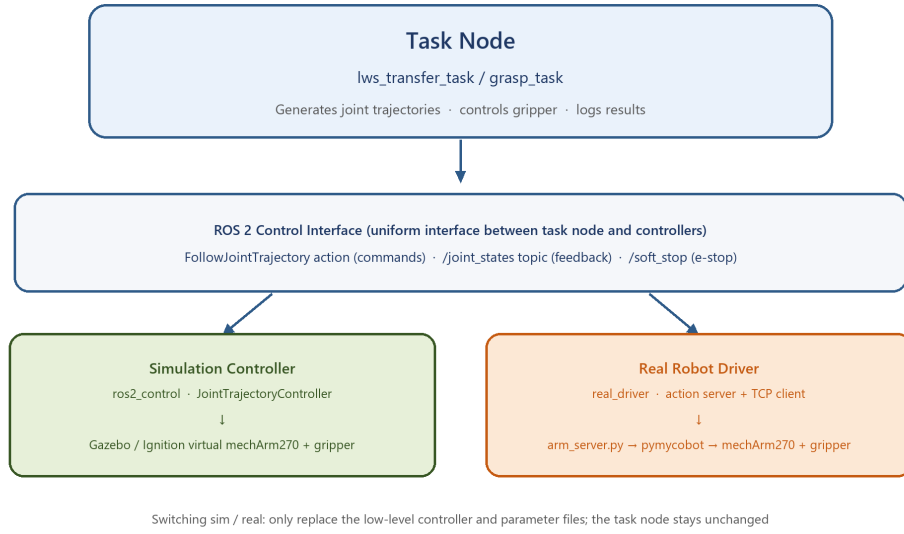


Figure 11: Overall system architecture.

8.2 Principles of Simulation Program

8.2.1 Task Node

The task node is the core of the simulation. Its main workflow is: home → move to above point A → descend → close gripper → lift → move to above point B → descend → open gripper → home. The workflow loops according to the `repeat_count` parameter. Odd cycles execute $A \rightarrow B$ transport while even cycles execute $B \rightarrow A$ transport. Therefore the object remains physically continuous without manual reset between cycles.

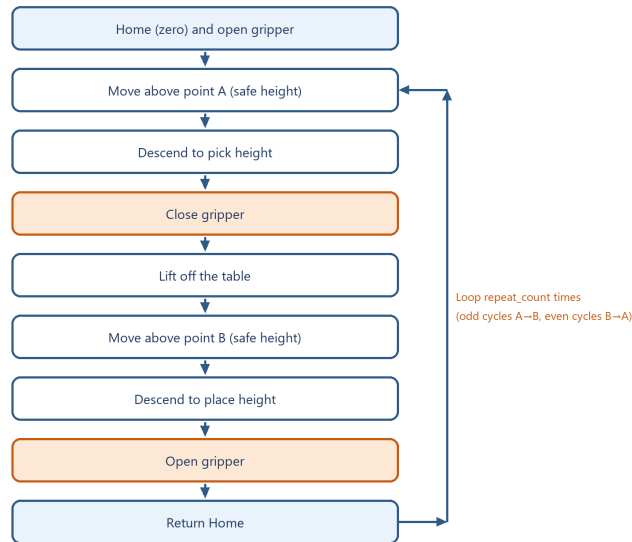


Figure 12: Workflow of the fixed-point pick-and-place task.

The node embeds forward-kinematics (FK) and numerical inverse-kinematics (IK) modules. FK builds a static transformation chain based on the origins of the six joints defined in the URDF, then superimposes the rotation of each joint around its own z -axis to obtain the pose of the end link `link6` and the gripper base `gripper_base`. The fingertip position is acquired by offsetting

`gripper_base` forward along the tool approach axis by `tool_tip_offset`. The IK solver adopts the damped least-squares method with a numerical Jacobian. It uses fixed and random seeds as candidate initial solutions, iteratively solving for joint angles that move the fingertip to the target point while keeping the tool axis vertically downward. Convergence is achieved when the position error is less than 2 mm and the tool-axis inclination is below approximately 15° . If none of the seeds converge, the target is marked unreachable and task startup is rejected. In addition, `align_jaw` fine-tunes the sixth joint via finite-difference Newton iteration to align the gripper opening direction with the coordinate axes of the object surface, preventing the object from being pushed away during oblique grasping.

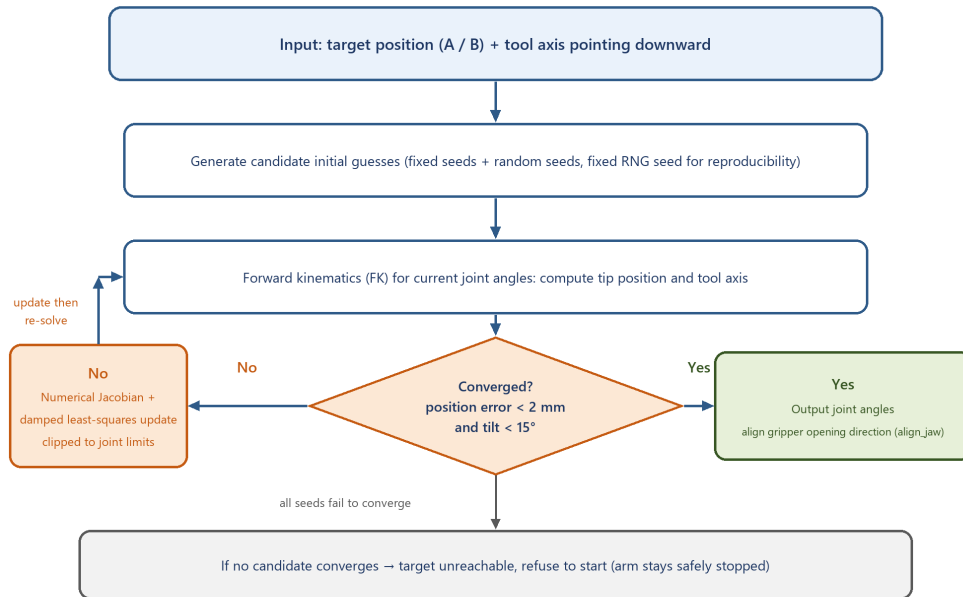


Figure 13: Numerical inverse-kinematics solving workflow.

For safety verification, every joint target is compared one by one against the URDF joint limits before transmission. Out-of-bound targets trigger immediate error reporting and task termination. After task completion (including abnormal termination), the process exits actively to avoid conflicts between residual old nodes and new instances competing for the same controller. Contact physics in Gazebo is difficult to stabilize during simulation, so a `sim_attach` adhesion switch is provided. After successful grasping detection, velocity servo at 30 Hz makes the object follow the end-effector smoothly. Before release, the servo target switches to the placement point and the velocity is set to zero. This switch is set to `false` on the physical robot, where the physical gripper takes over grasping naturally. During program execution, timestamps and joint positions are continuously written into `trajectory.csv`, successive test results into `results.csv`, and error logs into `errors.log` for result reproduction and project acceptance.

8.2.2 Launch File and Controller Configuration

`lws_transfer.launch.py` is a single launch file responsible for loading the Gazebo world, spawning the robot, bringing up the `ros2_control` controllers, starting `robot_state_publisher` for state broadcasting, and launching the task node. The `OnProcessExit` event ensures the execution sequence: spawn the robot first, then start the controllers, and finally launch the task node. Parameters including `gui`, `cycles`, `world`, and `params` can be overridden via the command line, supporting headless operation and abnormal-scenario testing. `controllers.yaml` sets `update_rate` to 100 Hz and defines two `JointTrajectoryControllers`: `arm_controller` for the six robotic joints and `hand_controller` for the gripper. The trajectory goal tolerance is

moderately relaxed to avoid misjudging small steady-state errors in the simulation position loop as timeouts.

8.3 Principles of Physical Robot Program

8.3.1 Driver Node `real_driver.py` and TCP Communication

The physical-robot driver node exposes exactly the same interfaces as the `ros2_control` used in simulation: two action servers `/arm_controller/follow_joint_trajectory` and `/hand_controller/follow_joint_trajectory`, the `/joint_states` state topic, and the `/soft_stop` software emergency-stop topic. Hence the task node can run without any modification when switching from simulation to physical hardware. Instead of accessing serial ports directly, the driver communicates with `arm_server.py` on the on-board Raspberry Pi through two TCP connections. The motion connection handles motion commands (blocking until the target is reached), and the state connection performs state polling and emergency stop; the two connections do not block each other. `arm_client.py` is a pure standard-library client for `arm_server`. It adopts the protocol of “one JSON object per line with blocking reply waiting”. Operations such as `goto`, `get_angles`, `gripper_value`, and `stop` are encapsulated as synchronous calls with automatic reconnection and two retry attempts.

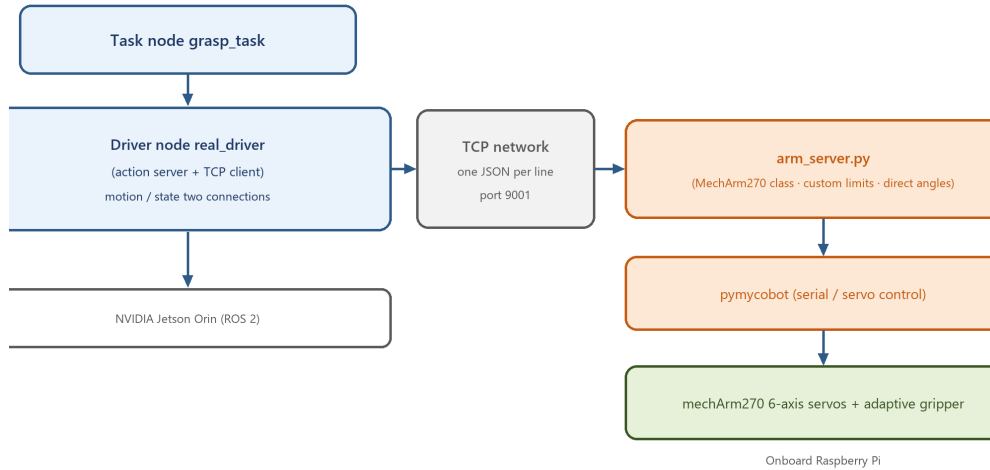


Figure 14: TCP communication architecture for the physical robot.

On the safety side, URDF joint limits are checked before each action target is sent, and the on-board controller re-verifies against physical hardware limits. After motion completion, the position residual is evaluated. If the residual exceeds the `goal_tol_deg` threshold, the task is marked failed and the task node enters the safe-return branch. The emergency stop halts motion only, without releasing servo torque, preventing the arm from falling. Gripper opening and closing are mapped from gripper joint angles into two discrete states. The driver judges whether the object is truly grasped by reading the gripper closing value. If the reading is outside the clamping range, the gripper automatically releases and retries grasping once. If the retry still fails, an empty grasp or blocked grasp is detected and the task terminates.

8.3.2 Teaching Points and Continuous Pick-and-Place Script

The physical robot does not generate targets via inverse kinematics; instead, teach-and-replay is applied. First, the joint angles of grasp point *A* and placement point *B* are taught via the `teach` command. Then `kinematics.py` (using identical chain parameters as the simulation) performs forward-kinematics calculation and writes the results into `real.yaml` as `at_a_joints_deg` and `at_b_joints_deg`. The task script `real_cycle_5_home_fast3_FINAL_SUCCESS.py` hardcodes

six verified poses as angle lists: HOME, A_SAFE, A_CLEAR, A_PICK, B_CLEAR, and B_PLACE. It continuously executes five pick-and-place cycles of $A \rightarrow B \rightarrow A \rightarrow B \rightarrow A$ and returns to HOME after each cycle. The script defaults to `execute:=false` for dry-run mode. Motion commands are only sent when `execute:=true` is explicitly passed, to prevent unintended movements. `real_grasp.launch.py` starts the physical-robot driver node first, and delays launching the task node until the driver finishes connection establishment.

9 GitHub Project Management

The experimental code and related materials are organized in the GitHub repository `Experiment_2`. The repository is structured into two directories: `simulation` and `real_robot`. It maintains README documents, source code, configuration files, logs, and experimental screenshots for reviewing the experimental workflow and the version-modification history.

GitHub repository: https://github.com/tuantuan01310304-stack/Experiment_2

10 Problem Analysis and Limitations

This experiment implements fixed-point pick-and-place tasks, so the positions of target objects must be predefined. The system cannot automatically adjust grasping points according to variations in the actual object position. Numerical inverse kinematics near the workspace boundary may also be affected by initial guesses and target reachability. In addition, physical-robot experiments require stable relative positions among the robot base, workbench, and initial object placement; otherwise, deviations will occur in the fixed-point parameters.

Future improvements can integrate camera visual localization, automatic calibration, collision planning, and more general grasp-pose generation based on the existing ROS 2 control architecture, enabling the robotic arm to adapt to changing target positions.

11 Experiment Conclusion

This experiment implements the fixed-point pick-and-place workflow from Gazebo / Ignition simulation to the physical `mechArm270` robotic arm. The system realizes robot homing, safe approach, descent, gripper control, lifting, transportation, placement, and return-to-home motions. It can record trajectories, experimental results, and exception information. Five consecutive task tests and unreachable-target tests were completed in simulation. For physical-robot tests, five A -to- B pick-and-place trials were performed with a 100% success rate and an average single-cycle runtime of 35.998s. Meanwhile, out-of-limit joint targets can be correctly rejected by the controller with an ABORTED status, which demonstrates that the system has basic safety-protection capability. Through this experiment, the authors have gained deeper familiarity with ROS 2 trajectory control, Gazebo simulation, robotic-arm inverse kinematics, physical-robot driver development, and the general pipeline of migrating algorithms from simulation to physical robots.

12 GitHub Repository and Commit History

The complete source code, configuration files, experiment logs, screenshots, and project documentation are available in the following GitHub repository:

https://github.com/tuantuan01310304-stack/Experiment_2?utm_source=chatgpt.com

The repository was updated through multiple commits during the experiment rather than being submitted as a single final upload. Figure 15 shows the commit history of the project.

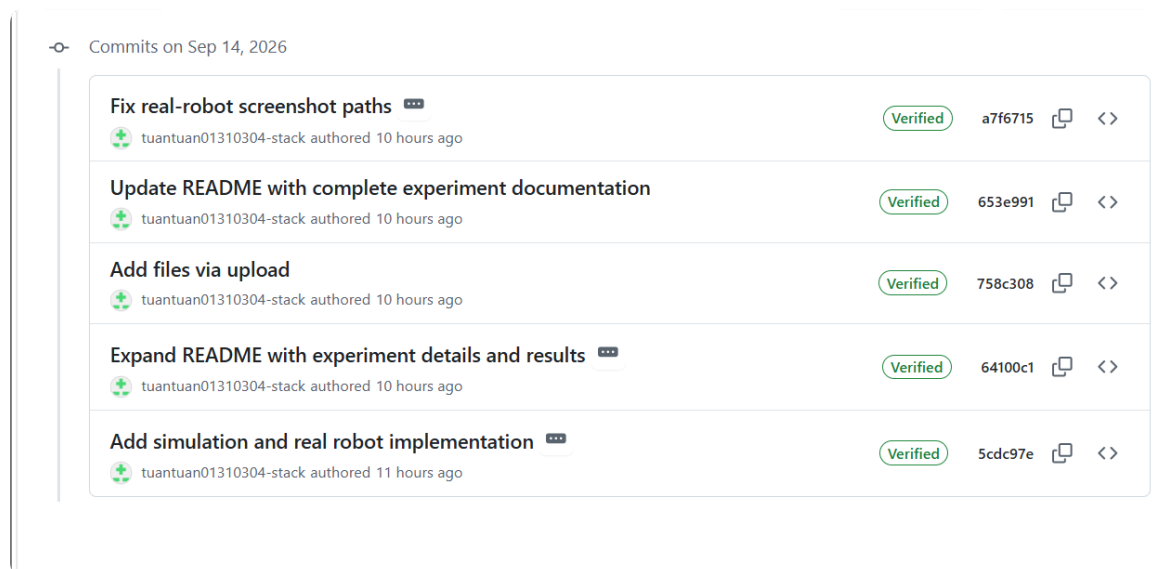


Figure 15: GitHub commit history of the Experiment_2 repository.